

01_Data_StructureMonotonic_QueueSliding_Window 单调队列 + 滑动窗口_板子

单调队列

适用场景：求长度为 k 的滑动窗口内的最大值或最小值（经典题：洛谷 P1886，LeetCode 239）。

核心数据结构：std::deque（双端队列），队列里存的是元素的“下标”而不是“值”（方便判断元素是否滑出了窗口）。

记忆口诀：“去老（队首出界）->去弱（队尾维护单调性）->新人入队->收集答案”

以下是求滑动窗口最大值的模板（维持队列单调递减）：

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    int n = nums.size();
    vector<int> res;    // 存储每个窗口的最大值
    deque<int> dq;      // 单调队列，存储元素的【下标】

    for (int i=0; i<n; i++) {
        // 1. 维护队首（出界）：如果队首元素的下标已经不在当前窗口 [i-k+1, i] 内，弹出队首
        // 队首元素下标 == i - k 时，说明它刚刚滑出窗口
        if (!dq.empty() && dq.front() == i - k) dq.pop_front();

        // 2. 维护单调性（队尾卷王机制）：
        // 我们要求最大值，所以要把前面那些【比当前元素小】且【比当前元素旧】的元素都淘汰掉
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();

        // 3. 入队：将当前元素的下标加入队尾
        dq.push_back(i);

        // 4. 记录答案：当窗口完全形成后（即 i >= k - 1），队首元素就是当前窗口的最大值
        if (i >= k - 1) res.push_back(nums[dq.front()]);
    }
    return res;
}
```

滑动窗口

适用场景：求满足某种条件的连续子数组（如求和大于等于 S 的最短子数组，或无重复字符的最长子串）。

核心思想：维护一个左指针 left 和右指针 right，像一条毛毛虫一样交替向前爬行。

这是一个非常通用的变长滑动窗口模板：

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

int slidingWindowTemplate(vector<int>& nums, int target) {
    int n = nums.size();
    int left = 0;    // 左指针
```

```

int min_len = INT_MAX; // 记录最短长度，初始设为无穷大
int window_sum = 0;    // 窗口内的具体状态
for (int right = 0; right < n; right++) {
    // 1. 右指针主动吃进元素，更新窗口状态
    window_sum += nums[right];
    // 2. 判断窗口是否已经【达标/合法】（比如和 >= target 了）
    // 一旦达标，我们就尝试缩小窗口，看看能不能更短！
    while (window_sum >= target) {
        // 3. 此时窗口是【合法】的，赶紧记录当前的短度！
        // 注意：更新答案的代码在 while 里面！
        min_len = min(min_len, right - left + 1);
        // 记录完之后，尝试把左边的元素踢出去，看剩下的还达不达标
        window_sum = nums[left];
        left++;
    }
}
// 如果 min_len 还是无穷大，说明压根没找到合法的窗口，返回 0
return min_len == INT_MAX ? 0 : min_len;
}

```